



**National University of Computer and Emerging Sciences
Islamabad Campus**

CY-2006

Operating Systems

Semester Project: Kernel Wrapper Function

Kernel version: Linux 4.15.0
VM: Ubuntu 18.04 on VirtualBox
System Call: wait()

Submitted by: Sarita Sangrez, Amna Ali

Roll numbers: 23i-2088, 23i-2067

Date: 30th November 2025

Contents

Introduction	3
Selected System Call: wait()	3
Purpose and Importance in Linux.....	3
System Setup and Environment	3
Virtual Machine Configuration	4
Development Environment Preparation	4
Wrapper Creation	4
Implementing the Wrapper Logic.....	4
Updating the System Call Table.....	5
Registering the Function Header	6
Kernel Compilation	6
Pre-Compilation Configuration Fix.....	6
The Build Process.....	6
Kernel Installation	6
Testing the Wrapper	7
Comparison Based Testing.....	9
Challenges faced and Solutions	10
Conclusion.....	11

Introduction

The Operating System kernel acts as the core interface between a computer's hardware and its processes. One of the fundamental mechanisms by which user-level applications interact with this privileged kernel core is through **System Calls**. A system call allows a program to request a service from the kernel, such as opening a file, creating a new process, or managing memory.

The objective of this project is to gain hands-on experience with Linux Kernel programming by implementing a **Custom System Call Wrapper**. Rather than modifying the deep internal logic of the kernel, which risks system instability, we are implementing a "wrapper" function. This wrapper intercepts calls to an existing system call, executes custom logic (such as logging or validation), and then passes execution to the original kernel function.

Selected System Call: wait()

For this project, the group selected the wait() system call. In the Linux kernel architecture, while user-space applications typically invoke the function wait(), the standard C library (glibc) implements this by invoking the underlying kernel system call **sys_wait4**. This is the specific kernel function targeted for the wrapper implementation.

The wait() system call is a fundamental process synchronization mechanism used in UNIX-like operating systems. When a process calls wait(), the following sequence occurs:

1. **Suspension:** If the child process is still running, the execution of the parent process is suspended (blocked). The operating system moves the parent from the "Running" state to the "Waiting" state.
2. **State Change Detection:** The parent remains blocked until one of its child processes changes state—specifically, when a child terminates (exits) or is stopped by a signal.
3. **Status Retrieval:** Once the child terminates, the system call returns control to the parent. Crucially, it provides the parent with the **exit status** of the child.

Purpose and Importance in Linux

The wait() system call is not merely a convenience; it is a cornerstone of stable process management in Linux. Its importance stems from three critical roles:

- **Prevention of Zombie Processes:** When a child process finishes execution, it does not disappear immediately. It remains in the system process table as a "zombie" (defunct) process until its parent acknowledges its termination. The wait() call is the mechanism used to perform this acknowledgment (known as "reaping"). Without it, the process table would eventually fill up with zombie entries, exhausting available Process IDs (PIDs).
- **Resource Reclamation:** While the memory and open files of a terminated process are freed by the OS automatically, the kernel stack and the task_struct (the process descriptor) remain allocated until the parent calls wait(). This system call ensures that these final kernel resources are properly recycled.
- **Flow Control:** The wait() mechanism is essential for the operation of command-line shells (like Bash). When a user runs a command, the shell creates a child process to run that command and then calls wait(). This pauses the shell prompt, ensuring the user cannot type a new command until the previous one has finished.

System Setup and Environment

To ensure a safe development environment, all kernel compilation and testing were performed within a Ubuntu 18.04 Virtual Machine. The environment was specifically curated to match the requirements of the legacy Linux Kernel 4.15.

Virtual Machine Configuration

We utilized **Oracle VirtualBox** to host the development environment. The VM was configured with significant resources to handle the computationally intensive task of kernel compilation.

- **Hypervisor:** Oracle VirtualBox
- **Host OS:** Windows
- **Guest Operating System:** Ubuntu 18.04 LTS (Bionic Beaver)

Ubuntu 18.04 was selected because its default compiler toolchain (GCC 7) is natively compatible with Kernel 4.15. Newer distributions (like Ubuntu 22.04) use newer compilers that cause syntax errors with older kernels.

Development Environment Preparation

Before obtaining the kernel source, the Ubuntu guest system was prepared with the necessary build dependencies and development tools.

1. Essential Build Tools: The following packages were installed to support the configuration and compilation process:

- **build-essential:** Contains GCC, Make, and standard libraries.
- **libncurses-dev:** Required for the text-based kernel configuration menu (menuconfig).
- **bison & flex:** Parsers required for processing kernel code.
- **libssl-dev & libelf-dev:** Required for cryptographic signatures and ELF object file handling.

2. Kernel Source Acquisition: We downloaded the "vanilla" (unmodified) upstream kernel source code directly from the official Linux Kernel Archives.

- **Source URL:** <https://mirrors.edge.kernel.org/pub/linux/kernel/v4.x/linux-4.15.tar.xz>
- **Version:** 4.15 (Stable)

3. Configuration: To ensure the kernel supports the VM's virtual hardware, we copied the existing configuration from the running Ubuntu OS (/boot/config-\$(uname -r)) and updated it using make oldconfig. This provided a stable baseline configuration for our custom modifications.

Wrapper Creation

The implementation of the custom system call wrapper involved modifications to three specific files within the Linux kernel source tree. This process ensured that the custom logic was defined, registered, and mapped to the appropriate system call number.

Implementing the Wrapper Logic

File Modified: kernel/exit.c

The core logic of the wrapper was implemented in the kernel's exit handling code. We chose kernel/exit.c because it contains the original sys_wait4 implementation, allowing our wrapper to easily access internal helper functions like kernel_wait4.

We appended the following SYSCALL_DEFINE4 macro to the end of the file. This macro automatically handles the architecture-specific requirements for system call arguments.

Code Implementation:

- Member 1: 23i-2088

```
/* -----
CUSTOM SYSTEM CALL WRAPPER
Student ID: 23i-2088
Wraps existing sys_wait4
----- */
SYSCALL_DEFINE4(wait_232088, pid_t, upid, int __user *, stat_addr, int, options, struct rusage __user *, ru)
{
    /* print message to kernel ring buffer for logging (dmesg) */
    printk(KERN_INFO "_____ \n");
    printk(KERN_INFO "[232088] WRAPPER: sys_wait invoked by PID %d\n", current->pid);
    printk(KERN_INFO "[232088] TARGET: Waiting for child PID %d\n", upid);

    /* call original to pass control to the real kernel function */
    return kernel_wait4(upid, stat_addr, options, ru);
}
```

- Member 2: 23i-2067

```
/* Custom wrapper for wait4
* Roll No: 23i2067 */
SYSCALL_DEFINE4(wait4_23i2067, pid_t, upid, int __user *, stat_addr, int, options, struct rusage __user *, ru)
{
    struct rusage r;
    /* messages for logging*/
    printk(KERN_INFO "wrapper(23i2067): called by PID = %d\n", current->pid);
    printk(KERN_INFO "wrapper(23i2067): waiting for child PID = %d\n", upid);

    return kernel_wait4(upid, stat_addr, options, ru);
}
```

Updating the System Call Table

File Modified: arch/x86/entry/syscalls/syscall_64.tbl

To redirect the standard, wait functionality to our new wrapper, we modified the x86_64 system call table. This table functions as a dispatch map, linking unique system call numbers to specific kernel function pointers.

We located **entry 61** (associated with wait4) and updated the entry point to our custom function.

Modification:

- 23i-2088:

56	COMMON	clone	sys_clone/ptregs
57	COMMON	fork	sys_fork/ptregs
58	COMMON	vfork	sys_vfork/ptregs
59	64	execve	sys_execve/ptregs
60	COMMON	exit	sys_exit
61	COMMON	wait4	sys_wait_232088
62	COMMON	kill	sys_kill
63	COMMON	uname	sys_newuname

- 23i-2067

60	COMMON	exit	sys_exit
61	COMMON	wait4	sys_wait4_23i2067
62	COMMON	kill	sys_kill
63	COMMON	uname	sys_newuname

By replacing sys_wait4 with sys_wait_232088, we ensured that any time the OS receives a request for system call #61, it executes our logging wrapper first.

Registering the Function Header

File Modified: include/linux/syscalls.h

Finally, to ensure the kernel compiler could link the system call table to our new function definition, we added the function prototype to the global syscalls header file. This step prevents "implicit declaration" errors during the build process.

We added the following prototype just before the #endif directive at the end of the file:

Code Added:

```
asmlinkage long sys_wait_232088(pid_t upid, int __user *stat_addr, int options, struct rusage __user *ru);  
#endif
```

Kernel Compilation

Once the wrapper code was implemented and registered, the next critical phase was compiling the modified source code into a bootable kernel image. This process involves translating the C source code into machine code and linking the necessary modules.

Pre-Compilation Configuration Fix

Before initiating the build, we identified a potential build failure caused by the default Ubuntu configuration. The .config file (copied earlier from the host OS) contained a reference to CONFIG_SYSTEM_TRUSTED_KEYS, pointing to proprietary Canonical/Debian security certificates (debian/canonical-certs.pem). Since we are building a custom kernel without access to these private keys, the compilation would fail during the module signing phase.

Resolution: We manually edited the kernel configuration file to remove this dependency.

1. **Navigate to Source:** cd /linux-4.15
2. **Edit Configuration:** nano .config
3. **Modification:** We searched for CONFIG_SYSTEM_TRUSTED_KEYS and cleared the value within the quotes.

Configuration Change: CONFIG_SYSTEM_TRUSTED_KEYS=""

The Build Process

To compile the kernel, we utilized the GNU Make utility. Given the large size of the Linux kernel source, a single-threaded compilation would take several hours. To optimize this, we utilized the -j (jobs) flag combined with the nproc command to execute parallel build jobs.

Command Executed: make -j\$(nproc)

Kernel Installation

Following a successful compilation (no error codes seen), we installed the new kernel and updated the bootloader.

1. **Install Modules:** Copied the compiled drivers to /lib/modules.
sudo make modules_install
2. **Install Kernel:** Copied the kernel image to /boot and updated GRUB.
sudo make install
3. **Reboot:** By default, the Ubuntu bootloader (GRUB) prioritizes the kernel with the highest version number (in this case, the stock Kernel 5.4). So, we interrupted the boot sequence by accessing the **GRUB Boot Menu** (holding the Shift key during startup). From the "Advanced options for Ubuntu" submenu, we explicitly selected **Linux 4.15.0**.

Kernel Version:

```
File Edit View Search Terminal Help
```

```
sarita@sarita-PC:~$ uname -r  
4.15.0  
sarita@sarita-PC:~$
```

```
amna@amna-VirtualBox:~$ uname -a  
Linux amna-VirtualBox 4.15.0 #1 SMP Thu Nov 27 19:16:18 PKT 2025 x86_64 x86_64  
x86_64 GNU/Linux  
amna@amna-VirtualBox:~$ uname -r  
4.15.0
```

Testing the Wrapper

To validate the successful implementation of the `sys_wait_232088` and `sys_wait_23i2067` wrapper, we developed a user-space C application designed to explicitly trigger the modified system call.

The test program utilizes the `fork()` system call to create a parent-child process relationship. The logic is designed to force the parent process to enter the "Waiting" state, thereby invoking our custom kernel wrapper.

Program Logic:

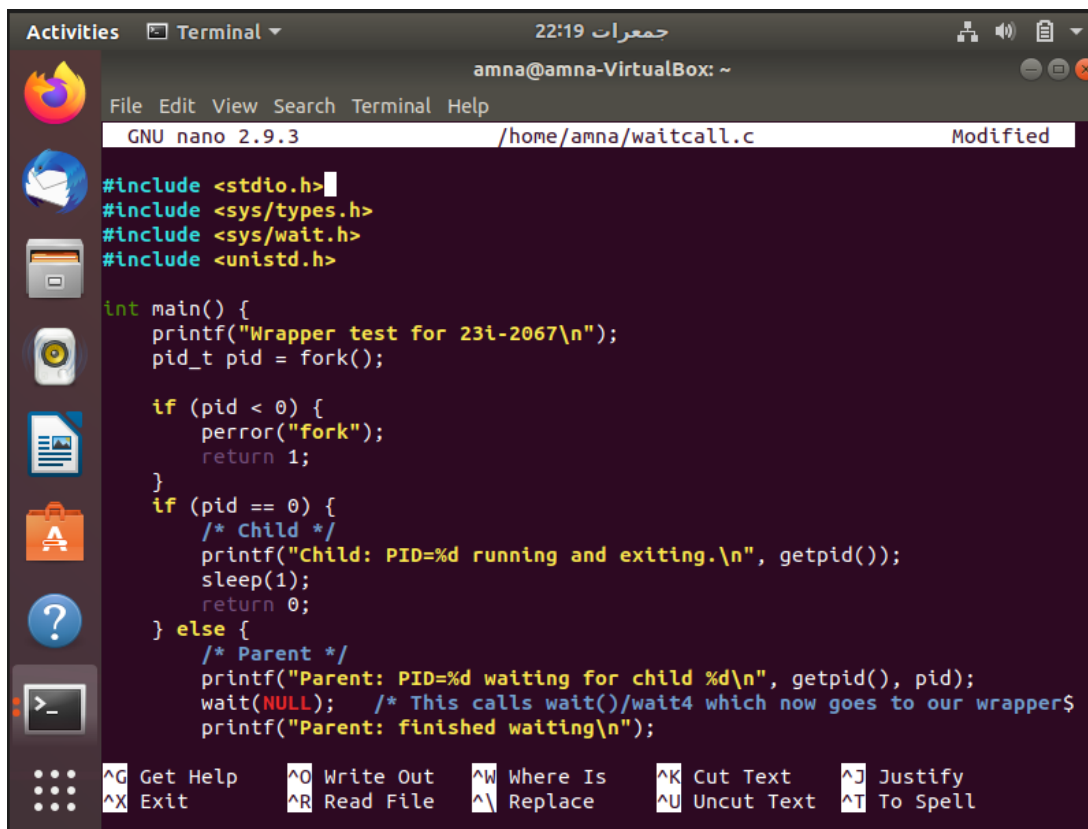
1. **Process Creation:** The program calls `fork()` to create a new child process.
2. **Child Execution:** The child process prints its Process ID (PID) to the user terminal and immediately exits.
3. **Parent Execution:** The parent process prints its own PID and the PID of the child it created. Crucially, it then calls `wait(NULL)`.
4. **The Trigger:** The standard C library translates this `wait()` call into the kernel system call `sys_wait4`. This is the precise moment our wrapper (`sys_wait_232088`) intercepts execution.

Program:

- Code on system of member 1: 23i-2088

```
GNU nano 2.9.3 testwrapper.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    pid_t pid;  
  
    printf("--- TESTING : Roll Number 23i-2088 ---\n");  
  
    // create child process  
    pid = fork();  
  
    if (pid < 0) {  
        perror("Fork failed");  
        return 1;  
    }  
  
    if (pid == 0) {  
        // --- child process ---  
        printf("[User Space] Child (PID: %d) is running...\n", getpid());  
        printf("[User Space] Child is exiting now.\n");  
        exit(42); // exit with code  
    } else {  
        // --- parent process ---  
        printf("[User Space] Parent (PID: %d) creates Child (PID: %d)\n", getpid(), pid);  
  
        // this triggers sys_wait4 -> sys_wait_232088  
        printf("[User Space] Parent calls wait()...\n");  
        wait(NULL);  
  
        printf("[User Space] Parent returned from wait().\n");  
        printf("--- End of Program ---\n");  
        printf("Check 'dmesg' for kernel logs\n");  
    }  
  
    return 0;  
}
```

- Code on system of member 2: 23i-2067



```
GNU nano 2.9.3 /home/amna/waitcall.c Modified
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    printf("Wrapper test for 23i-2067\n");
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        /* Child */
        printf("Child: PID=%d running and exiting.\n", getpid());
        sleep(1);
        return 0;
    } else {
        /* Parent */
        printf("Parent: PID=%d waiting for child %d\n", getpid(), pid);
        wait(NULL); /* This calls wait()/wait4 which now goes to our wrapper$
        printf("Parent: finished waiting\n");
    }
}
```

The application was compiled using the GNU Compiler Collection (GCC) and executed within the VM environment running the modified Linux Kernel 4.15. After the test program completed successfully in user space, the final verification was involved inspecting the kernel's internal message buffer. Since printk writes to the kernel ring buffer (not the standard terminal output), we used the dmesg command to retrieve the logs.

Output:

```
sarita@sarita-PC:~$ gcc testWrapper.c -o 1
sarita@sarita-PC:~$ ./1
--- TESTING : Roll Number 23i-2088 ---
[User Space] Parent (PID: 10123) creates Child (PID: 10124)
[User Space] Parent calls wait()...
[User Space] Child (PID: 10124) is running...
[User Space] Child is exiting now.
[User Space] Parent returned from wait().
--- End of Program ---
Check 'dmesg' for kernel logs
sarita@sarita-PC:~$ dmesg | tail -n 10
[ 970.670336] [232088] TARGET: Waiting for child PID -1
[ 970.671395]
[ 970.671398] [232088] WRAPPER: sys_wait invoked by PID 10123
[ 970.671400] [232088] TARGET: Waiting for child PID -1
[ 970.672373]
[ 970.672375] [232088] WRAPPER: sys_wait invoked by PID 9854
[ 970.672376] [232088] TARGET: Waiting for child PID -1
[ 998.855534]
[ 998.855536] [232088] WRAPPER: sys_wait invoked by PID 9854
[ 998.855538] [232088] TARGET: Waiting for child PID -1
sarita@sarita-PC:~$
```

```

amna@amna-VirtualBox:~$ gcc waitcall.c -o a
amna@amna-VirtualBox:~$ ./a
Wrapper test for 23i-2067
Parent: PID=25443 waiting for child 25444
Child: PID=25444 running and exiting.
Parent: finished waiting
amna@amna-VirtualBox:~$ dmesg | tail -n 10
[ 616.737675] wrapper(23i2067): called by PID = 25428
[ 616.737676] wrapper(23i2067): waiting for child PID = -1
[ 632.993681] wrapper(23i2067): called by PID = 2141
[ 632.993682] wrapper(23i2067): waiting for child PID = -1
[ 632.994688] wrapper(23i2067): called by PID = 25443
[ 632.994689] wrapper(23i2067): waiting for child PID = -1
[ 633.995508] wrapper(23i2067): called by PID = 2141
[ 633.995510] wrapper(23i2067): waiting for child PID = -1
[ 666.181255] wrapper(23i2067): called by PID = 2141
[ 666.181257] wrapper(23i2067): waiting for child PID = -1

```

Comparison Based Testing

Since we replaced the entry in the system call table, the original `sys_wait4` is no longer directly accessible from user space. Therefore, the test demonstrates that **both** direct syscalls and standard library calls are now successfully intercepted by our wrapper.

1. **Direct Invocation:** We used `syscall(61, ...)` to manually trigger the interrupt. This verifies that the kernel dispatch table correctly points to `sys_wait_232088`.
2. **Library Invocation:** We used the standard `wait()` function. This verifies that existing system utilities and user programs that rely on the standard library are also being intercepted and logged by our wrapper.

Program:

```

printf("\nTest 1: Triggering Wrapper with syscall 61...\n");
pid1 = fork();
if (pid1 == 0) {
    // child
    exit(10);
} else {
    // parent manually invokes syscall
    // arguments: pid, stat_addr, options, rusage
    long ret;
    ret = syscall(SYS_WAIT4, pid1, NULL, 0, NULL);
    printf("Syscall returned for Child PID: %ld\n", ret);
}

/*
Test 2: We call the standard libc wait(), which normally calls wait4 */
printf("\nTest 2: Triggering Standard Library wait()...\n");

pid2 = fork();
if (pid2 == 0) {
    // child 2
    exit(20);
} else {
    // parent calls standard wait
    wait(NULL);
    printf("Libc wait() returned for Child PID: %d\n", pid2);
}

printf("\n===== \n");
return 0;
}

```

Output:

```

sarita@sarita-PC:~$ nano test_comparison.c
sarita@sarita-PC:~$ gcc test_comparison.c -o 1
sarita@sarita-PC:~$ ./1
=====
TESTING: Original vs Wrapper
=====

Test 1: Triggering Wrapper with syscall 61...
Syscall returned for Child PID: 2208

Test 2: Triggering Standard Library wait()...
Libc wait() returned for Child PID: 2209

=====
sarita@sarita-PC:~$ dmesg | tail -n 20
[ 646.529115] [232088] WRAPPER: sys_wait invoked by PID 2211
[ 646.529116] [232088] TARGET: Waiting for child PID 2215
[ 646.530676]
[ 646.530678] [232088] WRAPPER: sys_wait invoked by PID 2211
[ 646.530679] [232088] TARGET: Waiting for child PID 2216
[ 646.535594]
[ 646.535595] [232088] WRAPPER: sys_wait invoked by PID 2211
[ 646.535595] [232088] TARGET: Waiting for child PID 2217
[ 646.547093]
[ 646.547096] [232088] WRAPPER: sys_wait invoked by PID 2218
[ 646.547097] [232088] TARGET: Waiting for child PID 2219
[ 646.596849]
[ 646.596850] [232088] WRAPPER: sys_wait invoked by PID 2218
[ 646.596850] [232088] TARGET: Waiting for child PID 2220
[ 646.666716]
[ 646.666720] [232088] WRAPPER: sys_wait invoked by PID 2211
[ 646.666721] [232088] TARGET: Waiting for child PID 2218
[ 652.705157]
[ 652.705160] [232088] WRAPPER: sys_wait invoked by PID 2045
[ 652.705161] [232088] TARGET: Waiting for child PID -1
sarita@sarita-PC:~$

```

Challenges faced and Solutions

Many problems were faced while compiling and setting up our own custom call wait() in linux kernel 4.15 on virtualbox ubuntu 18. Each challenge is given below along with the steps taken to fix them.

1. Error missing file during installation:

While installing modules “sudo make modules_install” there was an error saying modules.builtin not found. This was because kernel modules were not completely installed before modules so we ran the following commands to fix this issue

- The build environment was cleaned using “make clean”
- Then kernel was reconfigured using “make menuconfig”
- After this both the kernel and modules were reconfigured using “make -j\$(nproc)” and “make modules -j\$(nproc)”

2. Module signing failure:

Second issue was module signing failure due to missing signing keys. Default ubuntu config file contains “CONFIG_SYSTEM_TRUSTED_KEYS=“debian/canonical-certs.pem”” but this does not exist in vanilla linux kernel source. We solved it by going into .config file and setting

- CONFIG_SYSTEM_REVOCATION_KEYS=“”

3. Grub menu not displaying after installing kernel

The system would reboot directly without opening the grub menu to select kernel 4.15.0 so I edited the grub configuration file by

- GRUB_TIMEOUT_STYLE=menu
- GRUB_TIMEOUT=10

The configuration was enabled using “sudo update-grub” to ensure time is given to display grub by pressing “esc” on reboot.

4. Black screen after selecting the custom kernel 4.15.0

When I selected the Ubuntu kernel 4.15.0 from grub menu on reboot it just showed a black screen and nothing else. To fix this I rebooted the system again and after choosing 4.15.0 in grub menu I used `e` to edit it and added “nomodeset” to bypass. After successfully booting the updates were done again.

Conclusion

This project helped in learning something new like how to set up your own custom wrapper in a specific linux kernel version. It explained how system calls work. We created our own custom wrapper for system call `wait()` in linux kernel version 4.15 and learned how user-space processes communicate with kernel and how we can map and execute them. We manually modified some source files, mapped them into syscall table and used `printk` to keep logs of custom logic. We learned how kernel level logging works and how they can be viewed in system log buffer using the command “`dmesg`”.

This gave us an opportunity to understand the complete kernel compilation workflow from configuring files, building, installing modules, updating grub and then rebooting into desired kernel version. We also tested the wrapper using a simple forked `c` program which improved the concept of how parent-child processes verified that our custom wrapper was implemented successfully.